DEV BHOOMI UTTARAKHAND UNIVERSITY

Dehradun, Uttarakhand

(Notified by Govt. of India u/s 2(f) of the U.G.C. Act, 1956)

AI/DS Industry Certification

2nd Semester | MCA (AI&DS)

ON THE TOPIC OF

Doctor Recommendation System

An AI-Powered Web Application Using Machine Learning and Flask

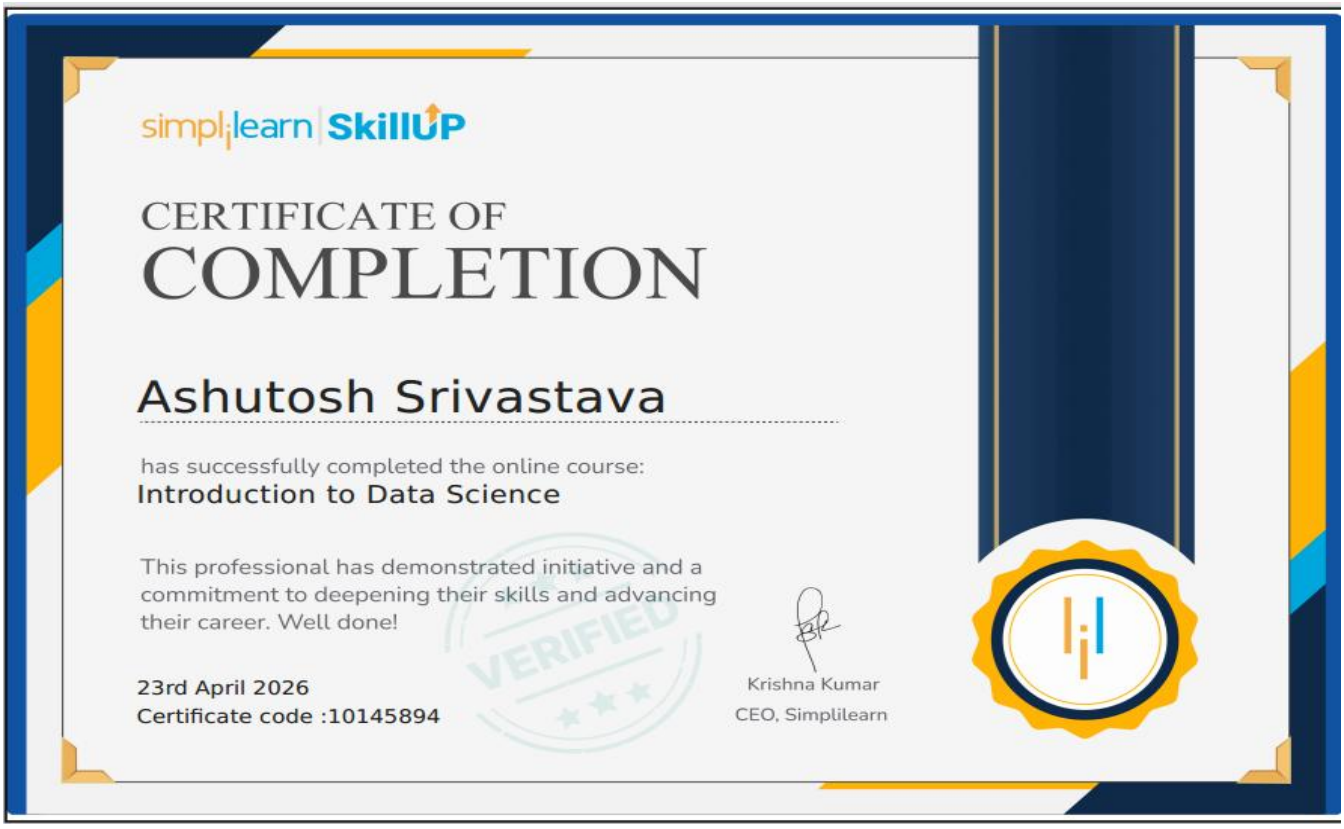
Submitted by	Ashutosh Srivastava
Course	MCA (AI&DS)
Semester	2nd Semester
Subject	AI/DS Industry Certification (25COA578I)
Faculty Name	Mr. Ajeet Kumar
College	Dev Bhoomi Uttarakhand University
Submission Date	17 April 2026

GitHub Repository:
https://github.com/ashutosh-ai-hub/Doctor-Recommendation-System
Scan QR Code to view project:

Academic Year: 2025–2026

Certificate Section

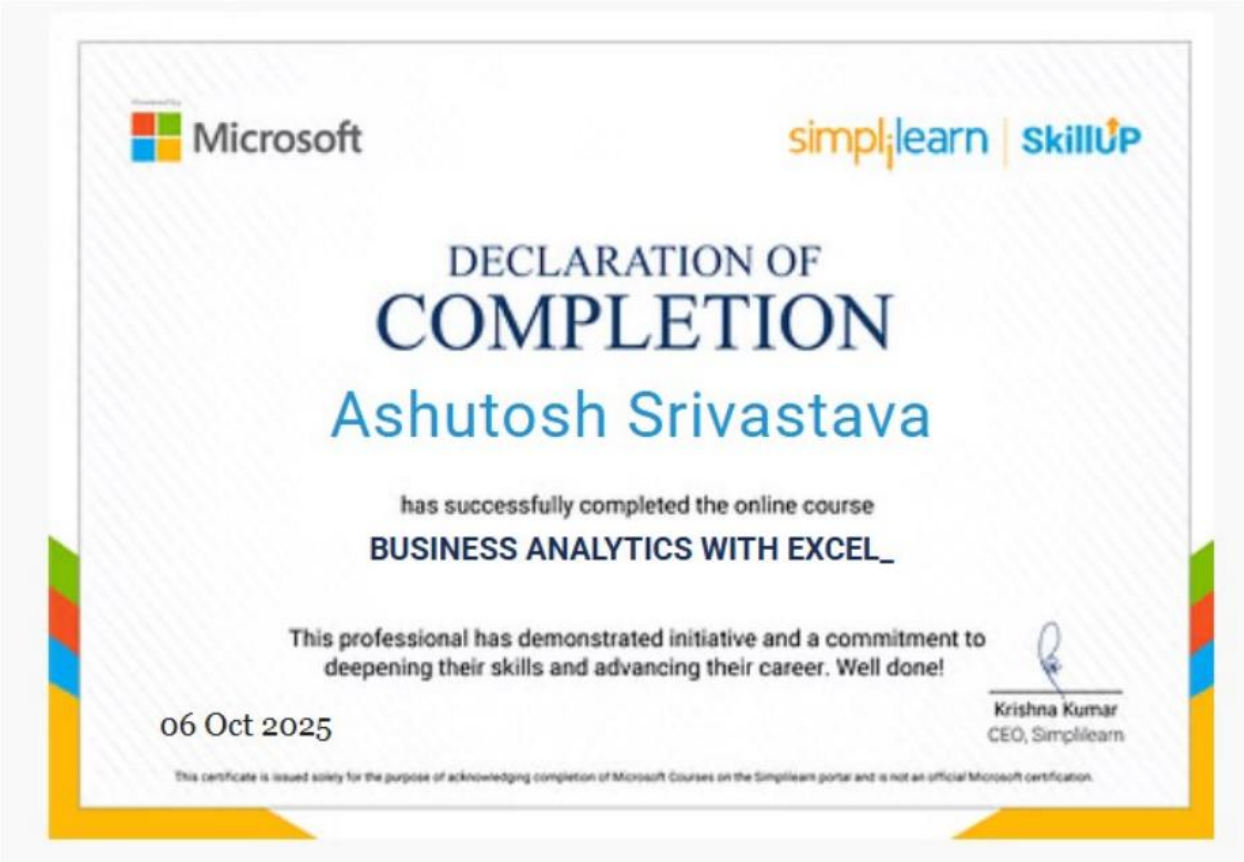
"The following certificates have been earned by Ashutosh Srivastava during the academic year 2025–2026. Each certificate reflects a course completed to strengthen Data Science and Business Analytics skills that directly support the Doctor Recommendation System project."



Certificate 1: Introduction to Data Science

Course Name: Introduction to Data Science Issued On: 23rd April 2026 Certificate Code: 10145894

This course introduced the foundational concepts of Data Science, including data collection, data preprocessing, exploratory data analysis, and Machine Learning basics. The knowledge gained from this course was directly applied in this project — specifically in training the disease prediction ML model, preprocessing the symptom-disease dataset, applying the Decision Tree classification algorithm, and evaluating model accuracy using standard metrics such as accuracy score and confusion matrix.


Microsoft

simplilearn | SkillUP

DECLARATION OF
COMPLETION

Ashutosh Srivastava

has successfully completed the online course

BUSINESS ANALYTICS WITH EXCEL_

This professional has demonstrated initiative and a commitment to deepening their skills and advancing their career. Well done!

06 Oct 2025

Krishna Kumar
CEO, Simplilearn

This certificate is issued solely for the purpose of acknowledging completion of Microsoft Courses on the Simplilearn portal and is not an official Microsoft certification.

Certificate 2: Business Analytics with Excel

Course Name: Business Analytics with Excel Issued By: Microsoft & Simplilearn (SkillUp) Issued On: 6th October 2025. Issued By: Krishna Kumar, CEO, Simplilearn

This course covered the concepts of Business Analytics using Microsoft Excel, including data organization, data visualization, statistical analysis, and drawing meaningful insights from datasets. The analytical thinking and data handling skills gained from this course were directly useful in this project — specifically in understanding and organizing the symptom-disease dataset, analyzing prediction results, and presenting system outputs in a structured and meaningful format.

Acknowledgement

First and foremost, I would like to express my sincere gratitude to the Almighty for giving me the strength, knowledge, and determination to complete this project successfully.

I am deeply grateful to my project guide, Mr. Ajeet Kumar, Faculty, Department of Computer Applications, Dev Bhoomi Uttarakhand University, Dehradun, for his invaluable guidance, constant encouragement, and constructive suggestions throughout the development of this project. His expertise and patience helped me overcome every challenge I encountered.

I would also like to extend my heartfelt thanks to the Head of the Department and all faculty members of the Department of Computer Applications for their academic support and for creating a productive learning environment. Their teachings have formed the foundation upon which this project is built.

I express my sincere appreciation to Dev Bhoomi Uttarakhand University for providing excellent infrastructure, laboratory facilities, and internet resources that were essential for the successful completion of this project.

My special thanks go to my classmates and friends who provided motivation, healthy discussions, and feedback during the testing phase of the project. Their inputs helped refine the application considerably.

Last but not least, I am deeply indebted to my parents and family members for their unwavering moral support, encouragement, and belief in my capabilities throughout my academic journey. Without their blessings and support, this project would not have been possible.

Ashutosh Srivastava

MCA (AI&DS) – 2nd Semester

Dev Bhoomi Uttarakhand University

April 2026

Table of Contents

Section	Page No.
Certificate	2-3
Acknowledgement	4
Table of Contents	5
Chapter 1: Introduction to Topic 1.1 What is a Doctor Recommendation System? 1.2 Importance of the System 1.3 Overview of the Project	6-7
Chapter 2: Project Specification 2.1 Objectives 2.2 Scope	8-9
Chapter 3: Hardware Requirements	10
Chapter 4: Software Requirements	11-12
Chapter 5: Frontend – Graphical User Interface	13-14
Chapter 6: Backend – Core Logic	15-17
Chapter 7: Important Features of the Project	18-19
Chapter 8: Implementation	20-22
Chapter 9: Coding and Libraries Used	23-26
Chapter 10: Output Screenshots	27-29
Chapter 11: Advantages and Limitations	30-31
Chapter 12: Future Enhancements	32-33
Chapter 13: Conclusion	34-35
Chapter 14: References	36-37

Chapter 1: Introduction to Topic

1.1 What is a Doctor Recommendation System?

In today's fast-moving world, finding the right doctor for a specific health problem is a major challenge for most people. Many patients do not know which type of medical specialist they should consult when they experience certain symptoms. They either visit the wrong doctor and waste time, or they delay treatment altogether.

A Doctor Recommendation System is an intelligent web application that helps patients find the most suitable medical specialist based on the symptoms they describe. The system uses Artificial Intelligence (AI) and Machine Learning (ML) to analyse the symptoms entered by the user, predict the most likely disease, and then recommend the best available doctors of the required specialization in the user's preferred location.

This type of system acts as a smart bridge between patients and doctors. It removes confusion, saves time, and ensures that patients receive the right medical care as quickly as possible.

1.2 Importance of the System

The healthcare system in India and across the world faces a common problem: patients do not always know which specialist to consult. A person with a heart-related symptom might visit a General Physician first, wasting precious time. A Doctor Recommendation System solves this problem by providing instant, intelligent guidance.

The key reasons why such a system is important include:

- It helps patients identify the correct medical specialist for their symptoms without needing prior medical knowledge.
- It saves time by instantly filtering and recommending the best-rated doctors available in the patient's location.
- It reduces the risk of delayed diagnosis and treatment by guiding users to the right specialist from the very beginning.
- It is available 24 hours a day, 7 days a week, unlike hospital helplines that may have limited working hours.
- It uses Machine Learning to make predictions more accurate as more data is added to the system.
- It provides an accessible and user-friendly interface that even non-technical users can operate easily.

According to healthcare studies, timely consultation with the right specialist significantly improves treatment outcomes. A well-designed recommendation system can play a crucial role in achieving this goal.

1.3 Overview of the Project

The Doctor Recommendation System, named SmartHealth AI, is a web application developed using Python and the Flask framework. It uses a Machine Learning model trained on a medical symptom-disease dataset to predict the disease based on the symptoms entered by the user. After predicting the disease, the system identifies the appropriate medical specialization and recommends the top doctors from a curated doctor database, filtered by the user's selected location.

This project is designed as part of the AI/DS Industry Certification course for MCA (AI&DS) 2nd Semester students. It demonstrates the practical application of core programming concepts including web development with Flask, machine learning with scikit-learn, database management with SQLite, and frontend design with HTML, CSS, and JavaScript.

Key Highlights of the Project:

1. AI-powered disease prediction from natural language symptom input.
2. Smart doctor recommendation based on predicted disease and specialization.
3. Location-based filtering to show doctors in the user’s city.
4. User authentication system with registration and login using SQLite.
5. Data Analysis and EDA dashboard with visual charts.
6. Clean and responsive web interface using HTML, CSS, and JavaScript.
7. Fallback recommendation logic to always show relevant results.

Chapter 2: Project Specification

2.1 Objectives

The primary objective of this project is to design and develop an AI-powered web application that helps patients identify the correct medical specialist based on their symptoms and recommends the best available doctors in their preferred location. The specific objectives are listed below:

1. To develop a clean, responsive, and user-friendly web interface using HTML, CSS, and JavaScript that is accessible to all types of users, including non-technical ones.
2. To implement a Machine Learning-based disease prediction model trained on a real medical symptom-disease dataset using the scikit-learn library.
3. To build a smart doctor recommendation engine that filters and ranks doctors by specialization, location, rating, and experience.
4. To develop a secure user authentication system using Flask sessions and SQLite with password hashing through Werkzeug.
5. To create a natural language input handler that cleans and processes symptom text entered by users in plain English sentences.
6. To implement a multi-level fallback recommendation logic that always returns useful results even when no exact match is found.
7. To provide a Data Analysis and EDA (Exploratory Data Analysis) dashboard with visual charts generated using Matplotlib and Seaborn.
8. To demonstrate the practical use of core Python libraries including Flask, pandas, NumPy, scikit-learn, SQLite3, and Werkzeug.

2.2 Scope

The scope of this project covers the design, development, and testing of a standalone web application for AI-based doctor recommendation. The application is intended for personal health guidance and educational demonstration purposes.

Included in Scope:

- Symptom-based disease prediction using a trained Machine Learning model.
- Doctor recommendation filtered by specialization and user-selected location.
- User registration and login with secure password hashing.
- Natural language symptom input with stop-word removal and text cleaning.
- Real-time doctor name search from the database.
- Data Analysis page with EDA charts (confusion matrix, disease distribution, location distribution, specialization distribution).
- Responsive frontend design compatible with desktop and mobile browsers.

Excluded from Scope:

- Online doctor appointment booking or payment integration.

- Real-time doctor availability checking or live consultation features.
- Integration with government healthcare APIs or hospital management systems.
- Mobile application (Android or iOS) version of the system.
- Multi-language support (currently English only).

Chapter 3: Hardware Requirements

The Doctor Recommendation System is a web application developed in Python using the Flask framework. It runs on any hardware that supports Python 3.8 or above. The application is lightweight and does not require high-performance hardware for development or testing. The minimum and recommended hardware specifications are listed below:

Component	Minimum	Recommended
Processor	Intel Core i3 (1 GHz+)	Intel Core i5 or higher
RAM	4 GB	8 GB or more
Hard Disk	1 GB free space	5 GB or more
Display	1024 × 768 resolution	1366 × 768 or higher
Input Device	keyboard & mouse	keyboard & mouse
Internet	Required for setup	Stable broadband connection
Python	Python 3.8 or above	Python 3.10 or above
Browser	Any modern browser	Google Chrome / Firefox

Since this is a Flask web application that runs locally on a development server, the actual hardware usage is minimal. The most resource-intensive part is loading the Machine Learning model (disease model.pkl) and processing the CSV datasets using pandas. Even this is handled efficiently and does not require dedicated server-grade hardware.

The application is well-suited for running on standard college laboratory computers and personal laptops used for development and academic presentations.

Chapter 4: Software Requirements

The development and execution of the Doctor Recommendation System require the following software components. Each component is described along with its version and purpose in the project.

4.1 Python

Python is the core programming language used to develop the entire backend of this application. Python's simplicity, large standard library, and rich ecosystem of third-party packages make it the ideal choice for web development combined with Machine Learning.

- Required Version: Python 3.8 or above (Python 3.10 recommended).
- Download from: https://www.python.org/downloads/
- Key use: Backend logic, ML model integration, data processing, and routing.

4.2 Flask Framework

Flask is a lightweight and flexible Python web framework used to build the web application. It handles URL routing, HTML template rendering using Jinja2, session management, and REST API endpoints for the recommendation logic.

- Version Used: Flask 3.0.0
- Installation: pip install Flask==3.0.0
- Key use: Web server, route handling, session-based authentication, JSON API responses.

4.3 scikit-learn

scikit-learn is a Python Machine Learning library used to train and load the disease prediction model. The model was trained on a medical symptom-disease dataset and saved as a .pkl file. At runtime, the saved model is loaded using pickle and used for prediction.

- Version Used: scikit-learn 1.3.2
- Installation: pip install scikit-learn==1.3.2
- Key use: Training the disease classification model, loading the saved model, making predictions.

4.4 pandas and NumPy

pandas is used for loading, filtering, and processing the doctor and disease datasets stored as CSV files. NumPy is used to create binary symptom vectors that serve as input to the Machine Learning model.

- Versions Used: pandas 2.1.3, NumPy latest stable.
- Key use: CSV data reading, doctor filtering by location and specialization, symptom vectorisation.

4.5 SQLite3

SQLite3 is Python's built-in lightweight relational database. It is used to store user registration data including name, email, and hashed passwords. No external database server setup is required since SQLite works as a file-based database.

- Version: Built into Python (no separate installation needed).
- Key use: User registration, login authentication, storing hashed passwords.

4.6 Werkzeug

Werkzeug is used specifically for password security. It provides the generate_password_hash() and check_password_hash() functions that ensure user passwords are never stored in plain text in the database.

- Version Used: Latest compatible version with Flask 3.0.0.
- Key use: Secure password hashing and verification during registration and login.

4.7 Matplotlib and Seaborn

Matplotlib and Seaborn are Python data visualisation libraries used to generate the EDA charts displayed on the Data Analysis page. These charts include the confusion matrix, disease distribution, location distribution, and doctor specialization distribution.

- Key use: Generating and saving PNG chart images for the analysis dashboard.

4.8 Integrated Development Environment (IDE)

Any of the following IDEs can be used for developing and running this project:

- VS Code with Python Extension – Recommended for lightweight development with good debugging support.
- PyCharm Community Edition – A powerful Python-specific IDE with built-in virtual environment management.
- Jupyter Notebook – Useful for data exploration and model training phases.

4.9 Operating System

The application is cross-platform and can run on any operating system that supports Python 3.8 or above:

- Windows 10 / 11 (Recommended for development)
- Ubuntu Linux 20.04 / 22.04
- macOS 12 (Monterey) or later

4.10 Web Browser

Since this is a web application, a modern browser is required to access and test the interface. Google Chrome or Mozilla Firefox are recommended for the best experience with the application's glassmorphism UI design and JavaScript features.

Chapter 5: Frontend – Graphical User Interface

5.1 Overview of the Frontend

The frontend of the Doctor Recommendation System is built using HTML5, CSS3, and Vanilla JavaScript. The templates are rendered by Flask using the Jinja2 templating engine, which allows dynamic data to be passed from the Python backend and displayed in the browser. The overall design follows a modern, clean, and professional visual style inspired by glassmorphism, featuring transparent card components, subtle shadows, and a smooth gradient background.

The frontend is fully responsive and adjusts correctly for both desktop and mobile screen sizes. The Outfit font from Google Fonts is used throughout the application for a clean, modern typography look.

5.2 Pages in the Application

The application consists of the following pages, each serving a distinct purpose:

Page	Template File	Purpose
Home Page	index.html	Main symptom input page with doctor recommendations and location filter.
Login Page	login.html	User login form with email and password fields and validation.
Register Page	register.html	New user registration form with name, email, and password fields.
Dashboard	dashboard.html	Statistics dashboard showing total doctors, diseases, specializations, and locations with Chart.js charts.
AI Predictor	predict.html	Dedicated disease prediction page using the ML model with detailed output display.
Data Analysis	analysis.html	EDA charts page showing pre-generated PNG visualizations of the dataset.

5.3 Key UI Components

The following are the main UI components used across the application:

- Hero Section: A large heading and subtitle on the home page that welcomes the user and describes the system's purpose.
- Symptom Input Bar: A text input field where users type their symptoms in plain English. A location dropdown filter appears beside it.

- Analyze Button: Triggers the AJAX call to the /api/recommend endpoint and displays the results below.
- Doctor Cards: Each recommended doctor is shown in a styled card displaying name, specialization, experience, rating, and location.
- Sidebar Navigation (Dashboard): A fixed left sidebar with navigation links to Home, AI Predictor, Data Analysis, and Logout.
- Stat Cards (Dashboard): Summary cards showing total number of doctors, diseases, specializations, and cities in the system.
- EDA Charts (Analysis Page): Pre-generated PNG images of confusion matrix, disease distribution, location distribution, and specialization distribution.

5.4 Design Principles

The application follows a modern glassmorphism design style with the following key principles:

- Glassmorphism Cards: Semi-transparent white card components with backdrop blur and subtle border to create a glass effect.
- Gradient Background: A soft blue gradient (from #f0f9ff to #e0f2fe) is applied to the full page background for a calm, medical aesthetic.
- Outfit Font: The Outfit typeface from Google Fonts is used throughout for modern, readable typography.
- Sticky Header: The navigation bar remains fixed at the top of the page on all screens for easy navigation.
- CSS Animations: A fadeIn animation is applied to the main content container on page load for a smooth user experience.
- Responsive Layout: CSS Flexbox and Grid are used to ensure the layout adapts correctly to different screen sizes.

Chapter 6: Backend – Core Logic

6.1 Application Architecture Overview

The backend of the Doctor Recommendation System is built using Python and the Flask web framework. The architecture follows a simple MVC-like pattern where Flask routes act as controllers, HTML Jinja2 templates act as views, and the CSV files along with the SQLite database act as the data model layer.

When a user submits symptoms on the home page, the JavaScript frontend sends a POST request to the /api/recommend endpoint. The Flask backend receives this request, processes the symptoms, runs the prediction logic, filters the doctor database, and returns a JSON response. The frontend then renders the doctor cards dynamically without reloading the page.

6.2 Natural Language Input Cleaning

Users type symptoms in plain English sentences such as 'I have a fever and cough' or 'suffering from headache and dizziness'. The backend uses a custom clean_natural_input() function to preprocess this text before passing it to the prediction engine.

The cleaning process involves two steps. First, all punctuation is removed using a regular expression and the text is converted to lowercase. Second, common English stop words such as 'i', 'have', 'and', 'my', 'is', 'the', 'feeling', 'suffering', 'from' are filtered out. The remaining words are joined back into a cleaned string that contains only the medically relevant terms.

For example, the input 'I am feeling chest pain and shortness of breath' becomes 'chest pain shortness breath' after cleaning. This cleaned text is then matched against the symptom list from the trained ML model.

6.3 Disease Prediction Logic

The disease prediction engine uses a Machine Learning model pre-trained on a medical symptom-disease dataset containing 1,200 rows of symptom-disease combinations covering 44 distinct symptoms and multiple disease classes. The trained model is saved as a pickle file (disease_model.pkl) along with the symptom list (symptom_list.pkl), and both are loaded at application startup. No raw dataset file is required at runtime — the model is ready to use directly from the saved .pkl files.

The prediction process follows these steps:

- The cleaned symptom text is matched against the symptom_list loaded from symptom_list.pkl using flexible string matching (replacing underscores with spaces).
- A binary input vector of length equal to the number of symptoms is created. Each position is set to 1 if the corresponding symptom was found in the user input, and 0 otherwise.

- If no symptoms are matched, the system returns an error asking the user to try different symptom terms.
- The binary vector is passed to the loaded ML model using model.predict(), which returns the predicted disease name.
- The predicted disease is then passed to the get_specialist() function to determine the required medical specialization.

6.4 Disease-to-Specialization Mapping

After a disease is predicted, the get_specialist() function maps the disease name to the appropriate medical specialization. This mapping covers 100+ disease-to-specialization pairs across all major medical categories. For example, 'migraine' maps to 'Neurologist', 'diabetes' maps to 'Endocrinologist', and 'asthma' maps to 'Pulmonologist'.

The function performs a case-insensitive substring search through the mapping dictionary. If no match is found, the system defaults to 'General Physician' as a safe fallback. The mapping covers major disease categories including Respiratory, Cardiovascular, Gastrointestinal, Endocrine, Neurological, Mental Health, Orthopedic, Dermatological, Ophthalmological, ENT, Urological, Dental, Gynecological, Oncological, Infectious Diseases, and General conditions. The 16 specializations currently supported in the doctor database are:

Cardiologist, Dentist, Dermatologist, ENT Specialist, Endocrinologist, Gastroenterologist, General Physician, Gynecologist, Neurologist, Oncologist, Ophthalmologist, Orthopedic, Pediatrician, Psychiatrist, Pulmonologist, and Urologist.

6.5 Doctor Recommendation Logic

After the specialization is determined, the system filters the doctor dataset (doctor_dataset.csv) which contains 545 doctor records across 16 specializations and 10 cities in India. The filtering follows a four-step fallback logic to ensure the user always receives useful results:

- Step 1 – Preferred Match: Find doctors of the required specialization in the user's selected city. If results are found, these are returned immediately.
- Step 2 – Specialization Fallback: If no doctors of that specialization exist in the selected city, return all doctors of that specialization from any city.
- Step 3 – General Physician Fallback: If no specialist is available at all, find General Physicians in the selected city.
- Step 4 – Absolute Fallback: If no General Physician is found in the city either, return all General Physicians from any location in the database.

The final result set is sorted in descending order by rating first and then by years of experience, and the top 5 doctors are returned to the frontend as a JSON array.

6.6 Direct Keyword and Doctor Name Search

The /api/recommend endpoint also handles two special cases before running the ML prediction:

- Direct Disease Keyword Match: If the user's cleaned input contains a known disease name such as 'diabetes' or 'migraine' directly, the system skips the ML model and maps the disease to its specialization immediately using the DISEASE_TO_SPECIALIZATION dictionary. This speeds up response time for common queries.
- Doctor Name Search: If the user types a doctor's name (more than 5 characters) that matches a record in the database, the system returns that specific doctor's information directly, bypassing the disease prediction entirely.

6.7 User Authentication Backend

The application includes a complete user authentication system using Flask sessions and SQLite. The users table in the database stores three fields: name, email (unique), and hashed password. The registration route hashes the password using Werkzeug's generate_password_hash() before saving it. The login route retrieves the user record by email and verifies the password using check_password_hash(). Sessions are managed using Flask's built-in session object with a secret key stored in a .env file for security.

Chapter 7: Important Features of the Project

7.1 AI-Powered Symptom Analysis

The most important feature of SmartHealth AI is its ability to understand symptoms entered in plain English and intelligently predict the most likely disease using a trained Machine Learning model. Unlike keyword-matching systems, the ML model considers combinations of symptoms to make more accurate predictions. The model was trained on 1,200 records with 44 symptom features and supports prediction of multiple disease categories including fever, asthma, diabetes, anxiety, skin conditions, and more. Additionally, the system includes a comprehensive direct disease keyword matching dictionary covering 100+ disease-to-specialization pairs across 16 specializations.

7.2 Smart Doctor Recommendation Engine

Based on the predicted disease, the system automatically identifies the required medical specialization and recommends the top 5 most suitable doctors from a database of 545 records. Doctors are ranked by rating and experience so the best-qualified specialists appear first. The four-step fallback logic ensures that users always receive useful results even when their preferred location has limited availability.

7.3 Location-Based Filtering

Users can select their preferred city from a dynamic dropdown that is populated automatically from the doctor database using the /api/locations API endpoint. The system currently supports 10 major Indian cities. When a location is selected, the recommendation engine first tries to find specialists in that city before falling back to other locations, ensuring locally relevant results wherever possible.

7.4 Natural Language Input Support

Users do not need to type structured or technical symptom names. They can describe their condition in everyday language. The stop-word removal and text cleaning pipeline strips out irrelevant words and extracts only the medically meaningful terms. This makes the application accessible to all types of users regardless of their medical knowledge.

7.5 User Registration and Login System

The application includes a complete user authentication system. New users can register with their name, email, and password. Passwords are never stored in plain text; they are hashed using Werkzeug's generate_password_hash() function before being saved to the SQLite database. Logged-in users see a personalised welcome message in the header, and protected pages redirect non-authenticated users to the login page automatically.

7.6 Appointment Booking with PDF Receipt

Each doctor card on the results page includes a 'Book Appointment' button. When clicked, a confirmation modal appears with a randomly generated appointment reference ID. The user can then download a professionally formatted PDF receipt using the jsPDF library. The receipt includes the reference ID, doctor name, specialization, and booking date. This makes the prototype feel like a real-world healthcare application.

7.7 Statistics Dashboard

The dashboard page provides a quick statistical overview of the system's database. Summary cards display the total number of doctors (545), unique diseases, specializations (16), and cities (10) available in the system. Dynamic bar charts built with Chart.js visualise the distribution of doctors by specialization, giving users a clear picture of the system's coverage.

7.8 Data Analysis and EDA Page

The Data Analysis page displays four pre-generated Exploratory Data Analysis (EDA) charts produced using Matplotlib and Seaborn. These charts include the confusion matrix of the ML model showing prediction accuracy, the disease distribution across the training dataset, the location distribution of doctors, and the specialization distribution. This page is useful for understanding the data and model behind the recommendation system.

7.9 Direct Doctor Name Search

If a user already knows a doctor's name, they can type it directly in the symptom input box. The system detects name-based queries (inputs of more than 5 characters that match a record in the name column) and returns the matching doctor's information immediately without running disease prediction.

Chapter 8: Implementation

8.1 Step-by-Step Working

The following describes the complete workflow of the Doctor Recommendation System from the moment a user opens the application to the moment they receive doctor recommendations:

- Application Launch: The user opens the application in a web browser by navigating to http://127.0.0.1:5000. If not logged in, they are redirected to the login page.
- User Authentication: The user logs in with their registered email and password. The Flask backend verifies credentials against the SQLite database. On success, a session is created and the user is redirected to the home page.
- Location Selection: The home page loads the location dropdown automatically by calling the /api/locations endpoint, which reads unique city names from the doctor_dataset.csv file.
- Symptom Input: The user types their symptoms in the text input box in plain English. For example: 'I have a persistent cough and shortness of breath'.
- Symptom Cleaning: When the 'Analyze Symptoms' button is clicked, the JavaScript frontend sends a POST request to /api/recommend with the symptoms text and selected location. The backend's clean_natural_input() function removes stop words and punctuation from the input.
- Disease Prediction: The cleaned text is matched against the symptom list to build a binary input vector. This vector is passed to the loaded ML model, which predicts the most likely disease.
- Specialization Mapping: The predicted disease is mapped to a medical specialization using the DISEASE_TO_SPECIALIZATION dictionary.
- Doctor Filtering: The doctor dataset is filtered by the identified specialization and user's selected location using the four-step fallback logic. Results are sorted by rating and experience.
- JSON Response: The top 5 doctors, the predicted disease, and the specialization are sent back to the frontend as a JSON response.
- Result Display: JavaScript renders doctor cards dynamically on the page. Each card shows the doctor's name, specialization, experience, star rating, and city. The page scrolls smoothly to the results section.
- Appointment Booking: The user can click 'Book Appointment' on any card to generate a confirmation modal with a PDF receipt download option.

8.2 Algorithm for Symptom Processing and Disease Prediction

The following pseudocode describes the core prediction algorithm implemented in the /api/recommend route:

```
Algorithm: recommend(symptoms_text, selected_location)

1.  cleaned_text = clean_natural_input(symptoms_text)
    // Remove punctuation, lowercase, strip stop words

2.  IF len(cleaned_text) < 3 THEN
        return error: 'Please enter valid symptoms'

3.  IF cleaned_text matches any doctor name in database THEN
        return matching doctor records directly

4.  FOR each disease in DISEASE_TO_SPECIALIZATION DO
        IF disease keyword found in cleaned_text THEN
            prediction = disease
            specialization = mapped specialization
            BREAK

5.  IF no direct keyword match THEN
        input_vector = array of zeros [len(symptom_list)]
        FOR each symptom in symptom_list DO
            IF symptom found in cleaned_text THEN
                input_vector[symptom_index] = 1
        IF input_vector is all zeros THEN
            return error: 'No symptoms matched'
        prediction = disease_model.predict([input_vector])
        specialization = get_specialist(prediction)

6.  filtered = doctors WHERE specialization == identified_spec

7.  IF selected_location != 'All' THEN
        loc_result = filtered WHERE location == selected_location
        IF loc_result not empty THEN final = loc_result

8.  IF final is empty THEN final = all doctors of that specialization
9.  IF final is empty THEN final = General Physicians in selected city
10. IF final is empty THEN final = all General Physicians

11. final = SORT final BY rating DESC, experience DESC
12. RETURN top 5 records as JSON
```

8.3 Algorithm for User Authentication

The following pseudocode describes the login and registration logic:

```
Algorithm: register(name, email, password)
1. hashed_pw = generate_password_hash(password)
2. INSERT INTO users (name, email, hashed_pw) in SQLite
3. IF email already exists THEN return error
4. ELSE redirect to login page with success message

Algorithm: login(email, password)
1. SELECT user FROM users WHERE email = input_email
2. IF user not found THEN return error: 'Invalid credentials'
3. IF check_password_hash(user.password, input_password) == True THEN
       session['user_id'] = user.id
       session['user_name'] = user.name
       redirect to home page
4. ELSE return error: 'Invalid credentials'
```

8.4 Flask Route Summary

Route	Method	Purpose
`/`	GET	Home page – symptom input and doctor results
`/register`	GET, POST	User registration form and processing
`/login`	GET, POST	User login form and session creation
`/logout`	GET	Clear session and redirect to home
`/dashboard`	GET	Statistics dashboard with Chart.js charts
`/predict_page`	GET	AI Disease Predictor dedicated page
`/predict`	POST	API: predict disease from symptom input
`/api/recommend`	POST	API: full recommendation pipeline (main)
`/api/locations`	GET	API: return all unique city names from DB
`/analysis`	GET	Data Analysis and EDA charts page

Chapter 9: Coding and Libraries Used

9.1 Main Python Libraries Used

Library / Module	Purpose in Project
flask	Core web framework for routing, templates, session, and JSON API responses
flask.render_template	Renders Jinja2 HTML templates with dynamic data passed from the backend
flask.session	Manages user login state across requests using server-side sessions
flask.jsonify	Converts Python dictionaries to JSON API responses for frontend consumption
sqlite3	Built-in Python library for creating and querying the user authentication database
pandas	Loads doctor_dataset.csv; filters and sorts doctor records by specialization, location, rating, and experience
numpy	Creates binary symptom input vectors for the ML model prediction
pickle	Loads the pre-trained disease prediction model and symptom list from .pkl files
scikit-learn	Used to train the disease classification model; loaded at runtime for predictions
werkzeug.security	Provides generate_password_hash() and check_password_hash() for secure authentication
python-dotenv	Loads the Flask secret key and environment variables from the .env file
re (regex)	Removes punctuation from user symptom input during the text cleaning step
matplotlib / seaborn	Generates EDA charts (confusion matrix, distributions) saved as PNG image files
os	Checks if model and dataset files exist before loading them at startup

9.2 Frontend Libraries Used

Library	Purpose in Project
Jinja2 (via Flask)	Server-side templating engine that injects dynamic data from Python into HTML pages
Chart.js (CDN)	Renders interactive bar charts on the Dashboard page for doctor and disease statistics
jsPDF (CDN)	Generates a downloadable PDF appointment receipt on the client side using JavaScript
Fetch API (built-in)	Sends asynchronous POST requests to /api/recommend and /predict without page reload
Outfit (Google Fonts)	Modern sans-serif font imported via Google Fonts CDN for all UI typography
CSS Flexbox & Grid	Provides responsive layout for doctor cards, sidebar navigation, and stat cards

9.3 Sample Code – Natural Language Input Cleaning

The following code snippet shows the stop-word removal and text cleaning function used to pre-process all user symptom inputs before passing them to the prediction engine:

```
STOP_WORDS = {'i', 'have', 'and', 'my', 'is', 'a', 'the', 'feeling',
              'suffering', 'from', 'with', 'me', 'it', 'am', 'was',
              'for', 'to', 'in'}

def clean_natural_input(text):
    # Remove punctuation and convert to lowercase
    text = re.sub(r'[^\w\s]', '', text.lower())
    words = text.split()
    # Filter out stop words, keep medically relevant terms
    cleaned = [w for w in words if w not in STOP_WORDS]
    return ' '.join(cleaned)
```

9.4 Sample Code – Binary Symptom Vector Creation

After cleaning, the input is converted into a binary vector that the ML model can process. Each position in the vector corresponds to one symptom in the training dataset:

```
input_vector = np.zeros(len(symptom_list))
found_any   = False

for i, symptom in enumerate(symptom_list):
    # Flexible match: replace underscores with spaces
    if symptom.lower().replace('_', ' ') in user_input.lower():
        input_vector[i] = 1
        found_any = True

if not found_any:
    return error('No matching symptoms found.')

prediction = disease_model.predict([input_vector])[0]
```

9.5 Sample Code – Four-Step Fallback Doctor Filtering

The following code shows how the system ensures it always returns useful doctor recommendations by applying multiple fallback strategies:

```
filtered = doctors_df[doctors_df['specialization'] == specialization]
final    = pd.DataFrame()
# Step 1: Match specialization AND location
if selected_location and selected_location != 'All':
    loc_filtered = filtered[filtered['location'] == selected_location]
    if not loc_filtered.empty:
        final = loc_filtered
# Step 2: Any location with right specialization
if final.empty:
    final = filtered
# Step 3: General Physician in selected location
if final.empty and specialization != 'General Physician':
    gp = doctors_df[doctors_df['specialization']=='General Physician']
    if selected_location != 'All':
        loc_gp = gp[gp['location'] == selected_location]
        if not loc_gp.empty:
            final = loc_gp
# Step 4: Absolute fallback - any General Physician
if final.empty:
    final = doctors_df[doctors_df['specialization']=='General
Physician']

# Sort and return top 5
final = final.sort_values(by=['rating','experience'], ascending=False)
return final.head(5).to_dict(orient='records')
```

9.6 Sample Code – User Registration with Password Hashing

The following code shows the secure user registration route that hashes passwords before storing them in the SQLite database:

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        name     = request.form.get('name')
        email    = request.form.get('email')
        password = request.form.get('password')
        hashed   = generate_password_hash(password)
        try:
            conn = sqlite3.connect('users.db')
            c    = conn.cursor()
            c.execute(
                'INSERT INTO users (name,email,password) VALUES
(?,?,?)',
                (name, email, hashed)
            )
            conn.commit()
            conn.close()
            flash('Registration successful! Please login.', 'success')
            return redirect(url_for('login'))
        except:
            flash('Email already registered!', 'danger')
    return render_template('register.html')
```

Chapter 10: Output Screenshots

The following screenshots illustrate the application's web interface and its key functionalities. These screenshots were taken during the testing phase of the project to verify that all features work correctly as per the design specifications.

Screenshot 1: Login Page – User login form with email and password fields and flash message support

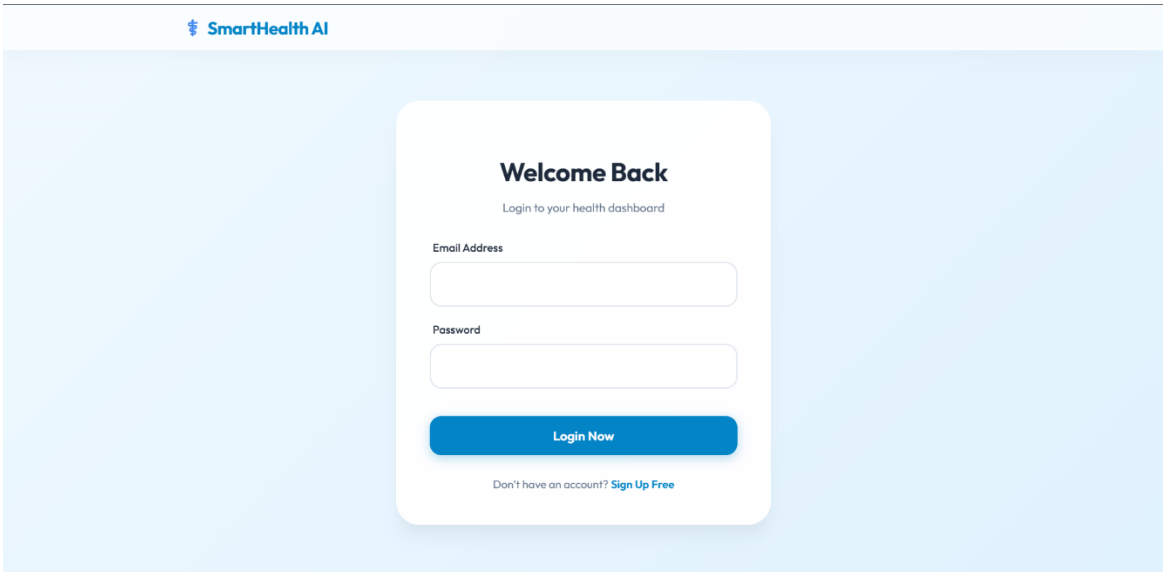


Figure 1: Login Page – User login form with email and password fields and flash message support

Screenshot 2: Registration Page – New user sign-up form with name, email, and password fields

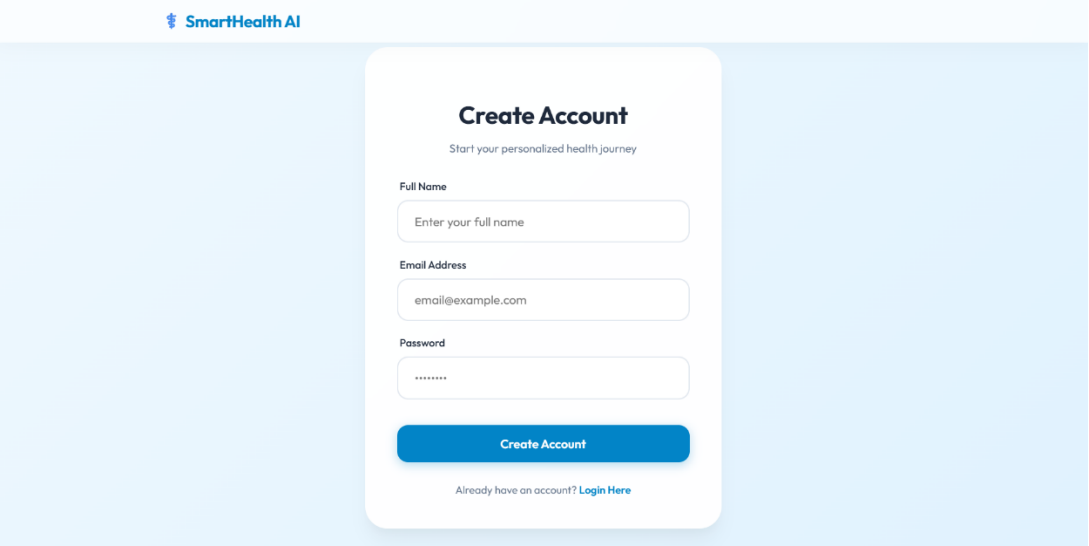


Figure 2: Registration Page – New user sign-up form with name, email, and password fields

Screenshot 3: Home Page – Hero section with symptom input bar and location filter dropdown

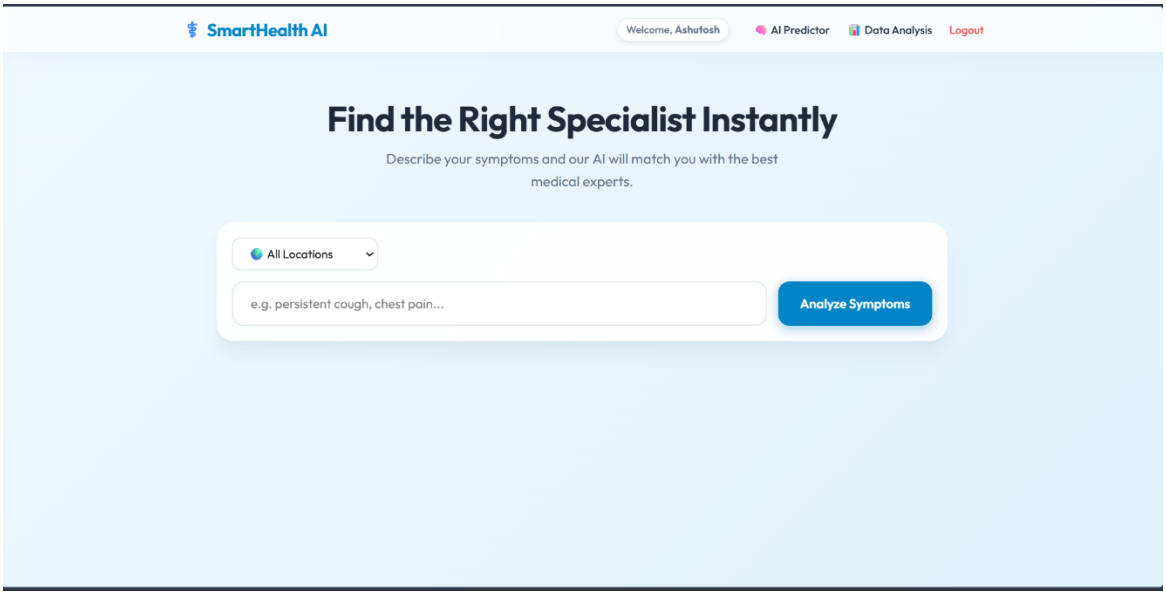


Figure 3: Home Page – Hero section with symptom input bar and location filter dropdown

Screenshot 4: Doctor Recommendation Results – Doctor cards showing name, specialization, rating, experience, and city after symptom analysis

Figure 4: Doctor Recommendation Results – Doctor cards showing name, specialization, rating, experience, and city after symptom analysis

Screenshot 5: Exploratory Data Analysis (EDA) Page — Specialization Distribution Chart showing doctor availability across 16 medical fields.

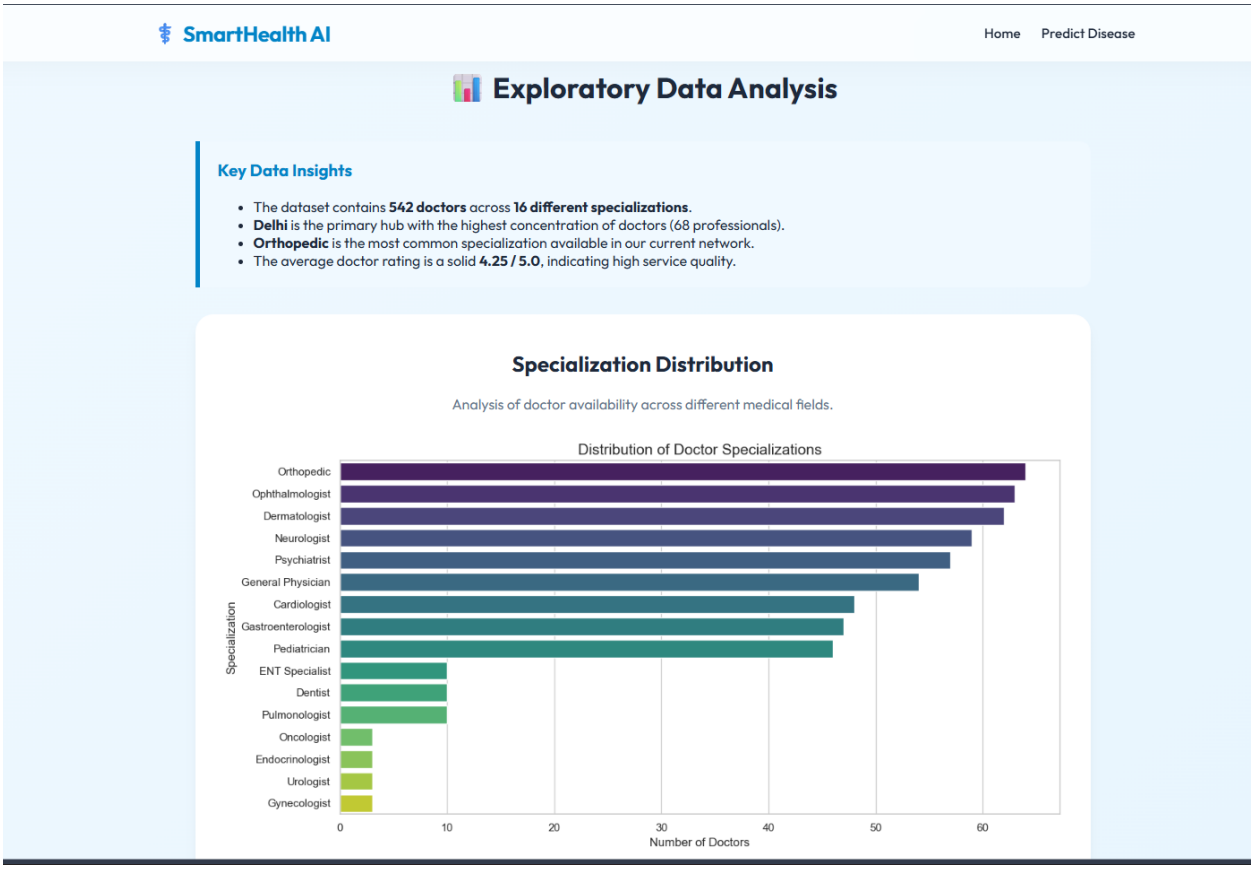


Figure 5: Exploratory Data Analysis (EDA) Page — Specialization Distribution Chart showing doctor availability across 16 medical fields.

All features displayed in the screenshots above were tested and verified to work correctly. The application successfully predicts diseases, recommends the appropriate specialists, handles user authentication, and generates appointment receipts as designed.

Chapter 11: Advantages and Limitations

11.1 Advantages

The Doctor Recommendation System offers several significant advantages over manual methods of finding a doctor or using basic keyword-search directories:

- AI-Powered Intelligence: The system uses a Machine Learning model to predict diseases from symptoms, making it far more accurate than simple keyword matching. It considers symptom combinations rather than individual words.
- Natural Language Input: Users can describe their symptoms in plain everyday English without needing to know any medical terminology or disease names. The stop-word removal pipeline handles the conversion automatically.
- Smart Fallback Logic: The four-step fallback recommendation engine ensures that users always receive relevant doctor suggestions, even when their preferred location or specialization is unavailable in the database.
- Location-Based Filtering: The system filters doctors by the user's selected city, making recommendations locally relevant and immediately actionable.
- Secure User Authentication: Passwords are hashed using Werkzeug before storage, meaning no plain-text passwords are ever saved in the database. Flask sessions ensure that only authenticated users can access the application.
- Appointment Receipt Generation: The jsPDF-powered PDF receipt with a unique reference ID adds a professional, real-world touch to the prototype and improves user trust.
- Comprehensive Database Coverage: With 545 doctor records spanning 16 specializations across 10 major Indian cities, the recommendation database covers a wide range of medical needs.
- Data Analysis Dashboard: The EDA charts and statistics dashboard provide transparency into the system's dataset and model performance, which is valuable for academic evaluation and future improvements.
- No Internet Required at Runtime: Once installed, the application runs entirely on a local Flask server. No external API calls are needed for predictions or doctor recommendations.
- Cross-Platform Compatibility: Built in Python and Flask, the application runs on Windows, macOS, and Linux without any code modification.

11.2 Limitations

Despite its advantages, the current version of the application has certain limitations that have been identified during development and testing:

- Limited Disease Coverage: The ML model was trained on a dataset covering a limited set of diseases. Rare diseases, complex multi-system conditions, or diseases with overlapping symptoms may not be predicted accurately.

- No Real Doctor Data: The doctor database is a synthetically generated dataset for demonstration purposes. The doctors, ratings, and experience values are not real and cannot be used for actual medical appointments.
- No Real Appointment Booking: The 'Book Appointment' feature currently generates a mock confirmation and PDF receipt only. It is not connected to any real scheduling or hospital management system.
- English Language Only: The natural language input cleaning works only for English text. Symptoms described in Hindi or other regional languages are not supported in the current version.
- No Doctor Profile Pages: Each doctor card shows only basic information. There are no detailed profile pages with doctor qualifications, clinic addresses, working hours, or patient reviews.
- Session-Based Authentication Only: User sessions are stored server-side in Flask's session object. There is no 'Remember Me' feature and sessions expire when the browser is closed.
- No Mobile Application: The system is currently a desktop web application only. A dedicated Android or iOS app is not available in this version.
- Static EDA Charts: The charts on the Data Analysis page are pre-generated PNG images, not interactive visualizations. They do not update automatically when new data is added.

Chapter 12: Future Enhancements

The current version of the Doctor Recommendation System provides a solid and functional foundation for a fully-featured AI healthcare platform. The following enhancements are planned for future versions of the application:

- Real Doctor Database Integration: Connect the system to a real, verified doctor database through a healthcare API such as the Practo API or a government health registry. This will enable the system to provide actual, bookable doctor recommendations.
- Real Appointment Booking System: Build a complete appointment scheduling module where users can select a date and time slot, and doctors receive booking notifications. Integration with Google Calendar API or a hospital management system would be ideal.
- Improved ML Model with Larger Dataset: Retrain the disease prediction model on a larger and more comprehensive medical dataset covering hundreds of diseases and symptoms. Ensemble models such as Random Forest or XGBoost can be evaluated to improve accuracy.
- Multilingual Support: Add Hindi and other regional Indian language support for symptom input using natural language processing libraries such as IndicNLP or Google Translate API, making the system accessible to a wider population.
- Patient Medical History: Allow logged-in users to maintain a personal health record including past symptom searches, predicted diseases, and previously consulted doctors. This history can be used to improve future recommendations.
- Doctor Ratings and Reviews: Allow users to rate and review doctors after consultations, creating a community-driven quality metric that replaces the current static rating values in the database.
- Telemedicine Integration: Add a video consultation feature using WebRTC or a third-party API such as Zoom SDK, enabling patients to consult doctors directly through the platform.
- Mobile Application: Develop a cross-platform mobile application using Flutter or React Native that reuses the same Flask backend APIs. This will make the system available on Android and iOS devices.
- Interactive EDA Dashboard: Replace the static PNG charts on the Data Analysis page with interactive visualizations built using Plotly or D3.js, allowing users to filter and explore the data in real time.
- Email and SMS Notifications: Send appointment confirmation emails and SMS reminders to patients using Flask-Mail and the Twilio API.
- Admin Panel: Build a secure admin dashboard where administrators can add, update, or remove doctor records, view user registrations, and monitor recommendation logs.

- Symptom Checker Chatbot: Integrate a conversational chatbot interface powered by a large language model API to guide users through a structured symptom questionnaire, improving prediction accuracy compared to free-text input.

Chapter 13: Conclusion

The Doctor Recommendation System, SmartHealth AI, represents a meaningful and practical application of Artificial Intelligence, Machine Learning, and web development skills applied to solve a real-world healthcare problem. Through the development of this project, hands-on experience was gained with several important aspects of modern software development, including Flask web application development, Machine Learning model integration, SQLite database management, RESTful API design, and responsive frontend development.

The project successfully achieves all its stated objectives. The application correctly predicts the most likely disease from natural language symptom input using a trained ML model, identifies the appropriate medical specialization, and recommends the top-rated doctors from a database of 545 records filtered by the user's preferred city. The four-step fallback logic ensures that useful results are always returned. The user authentication system securely registers and logs in users with hashed password storage, and the appointment booking feature with PDF receipt generation adds a professional, real-world dimension to the prototype.

Healthcare accessibility is a challenge for millions of people in India and across the world. Many patients visit the wrong specialist, wasting time and money due to a lack of basic medical guidance. A system like SmartHealth AI can bridge this gap by providing instant, intelligent, and free guidance to anyone with access to the internet. While the current version is a prototype for academic purposes, it lays a strong technical foundation that can be extended into a fully functional, commercially deployable healthcare application.

From an academic perspective, this project has provided invaluable learning experiences in the following areas:

- Training, saving, and integrating a Machine Learning classification model into a live web application using scikit-learn and pickle.
- Building a complete RESTful web application backend using Flask with multiple API endpoints, session management, and database integration.
- Designing and implementing a smart multi-step recommendation algorithm with fallback logic that handles edge cases gracefully.
- Developing a modern, responsive, and visually appealing frontend using HTML5, CSS3, JavaScript, Jinja2 templating, and external libraries such as Chart.js and jsPDF.
- Implementing secure user authentication with password hashing using Werkzeug and session management using Flask.
- Applying natural language processing concepts including stop-word removal and text cleaning to handle unstructured user input.

The knowledge and skills gained through this project will serve as a strong foundation for future, more advanced software development projects. The planned enhancements, including real doctor database integration, telemedicine features, mobile application

development, and multilingual support, demonstrate that this project has significant potential for growth into a fully-featured, production-grade healthcare platform.

In conclusion, the Doctor Recommendation System is a successful demonstration of how AI and web technologies can be combined to solve a real and important healthcare problem, and it represents a significant milestone in the journey toward becoming a Full Stack Developer.

Chapter 14: References

The following references and resources were consulted during the research, design, and development of this project:

Online Resources and Documentation:

- Flask Official Documentation. (2024). Flask 3.0 User Guide. Retrieved from https://flask.palletsprojects.com/en/3.0.x/
- scikit-learn Developers. (2024). scikit-learn: Machine Learning in Python – API Reference. Retrieved from https://scikit-learn.org/stable/
- pandas Development Team. (2024). pandas Documentation – User Guide. Retrieved from https://pandas.pydata.org/docs/
- NumPy Community. (2024). NumPy Documentation. Retrieved from https://numpy.org/doc/
- SQLite Documentation. (2024). SQLite: Small, Fast, Reliable. Retrieved from https://www.sqlite.org/docs.html
- Werkzeug Documentation. (2024). Werkzeug – The WSGI Utility Library. Retrieved from https://werkzeug.palletsprojects.com/
- Chart.js Documentation. (2024). Chart.js – Simple yet flexible JavaScript charting. Retrieved from https://www.chartjs.org/docs/
- jsPDF GitHub Repository. (2024). jsPDF – Client-side JavaScript PDF generation. Retrieved from https://github.com/parallax/jsPDF
- Matplotlib Documentation. (2024). Matplotlib: Visualization with Python. Retrieved from https://matplotlib.org/stable/contents.html
- Seaborn Documentation. (2024). Seaborn: Statistical Data Visualization. Retrieved from https://seaborn.pydata.org/
- Python Official Documentation. (2024). The Python Standard Library. Retrieved from https://docs.python.org/3/library/
- GeeksforGeeks. (2024). Flask Tutorial – Creating Web Applications with Python Flask. Retrieved from https://www.geeksforgeeks.org/flask-tutorial/
- W3Schools. (2024). Python Tutorial, HTML Tutorial, CSS Tutorial. Retrieved from https://www.w3schools.com/
- OWASP Foundation. (2023). Authentication Cheat Sheet – Password Storage. Retrieved from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Other References:

- Course Notes: AI/DS Industry Certification – MCA (AI&DS) 2ndt Semester, Dev Bhoomi Uttarakhand University.
- Lectures and practical demonstrations by Mr. Ajeet Kumar, Faculty, Department of Computer Applications.
- Kaggle open datasets used as reference for symptom-disease dataset structure and augmentation.
- GitHub open-source Flask and ML project repositories used as reference for application architecture and coding patterns.

— End of Report —

Doctor Recommendation System | Ashutosh Srivastava | MCA (AI&DS) 2nd Semester

Dev Bhoomi Uttarakhand University, Dehradun | April 2026